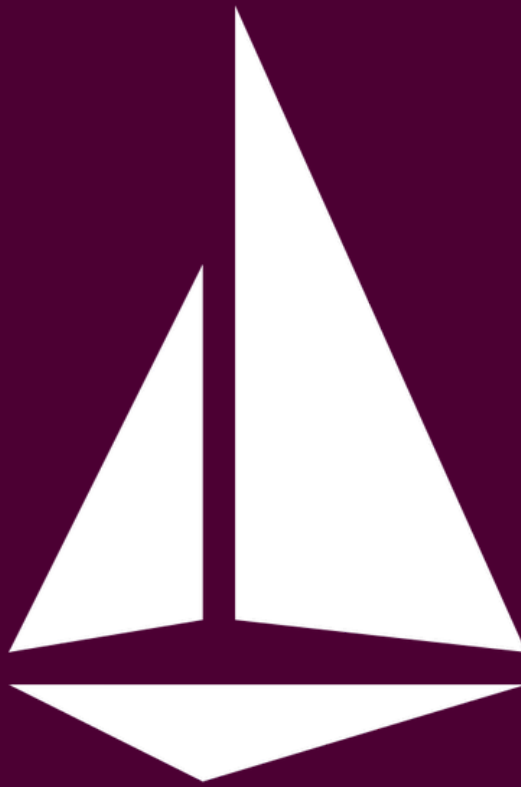


CNCF Project Focus

Episode #6



ISTIO

Service mesh for Kubernetes



Christian Dussol

ARC #2 - Observability

Measure. Trace. Control.



Episode #4
Prometheus



Measure



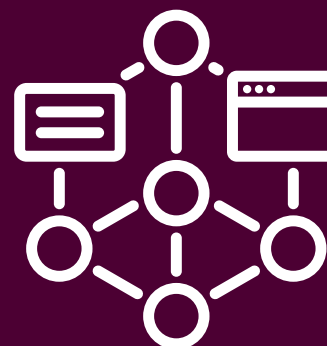
Episode #5
Open Telemetry



Trace



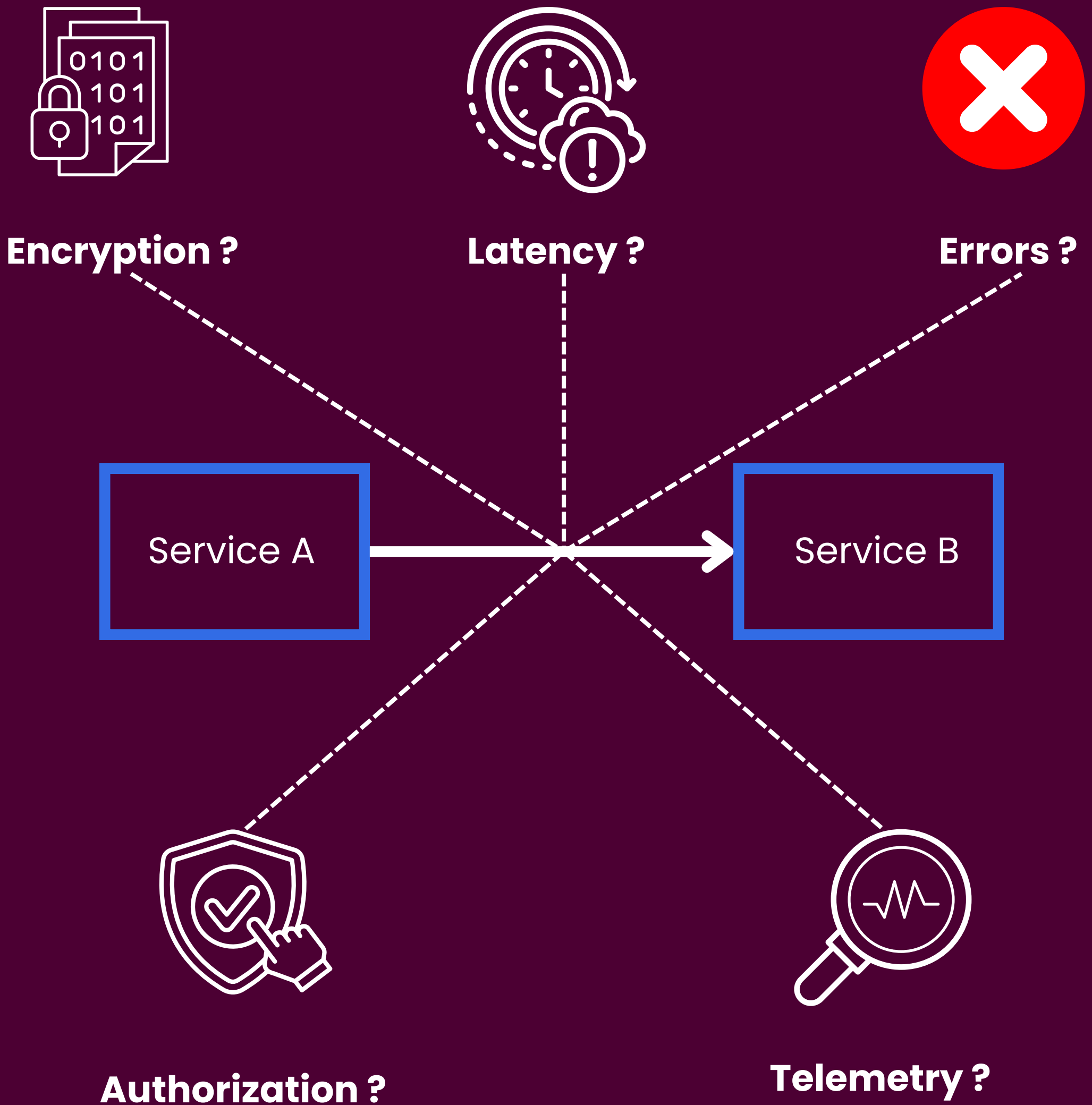
Episode #6
Istio



Control

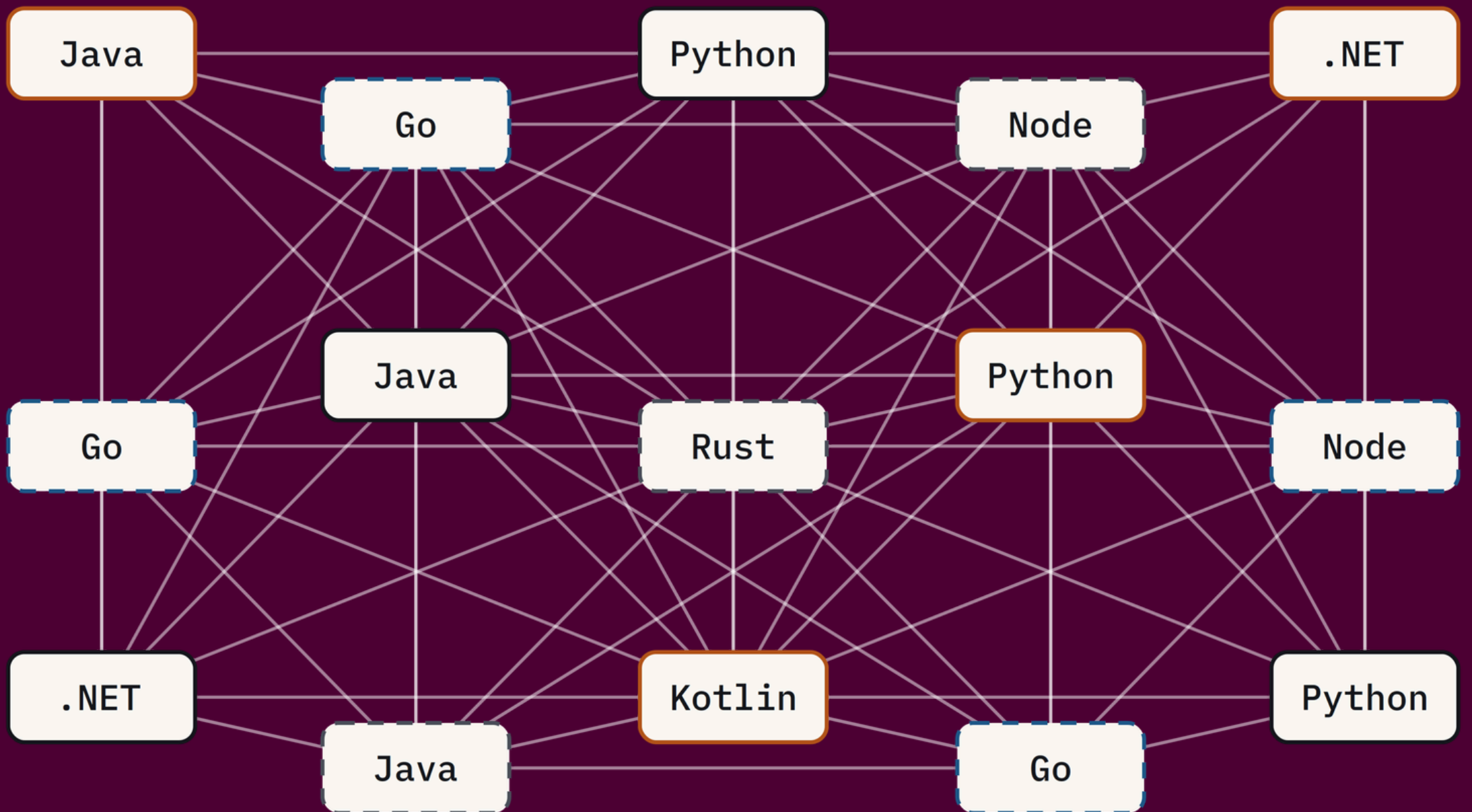


Anatomy of a service-to-service call



Service-to-service calls at scale

*50 services.
Up to 2,450 call paths.*



**Every team rebuilds retries,
timeouts and TLS. Its own way.**

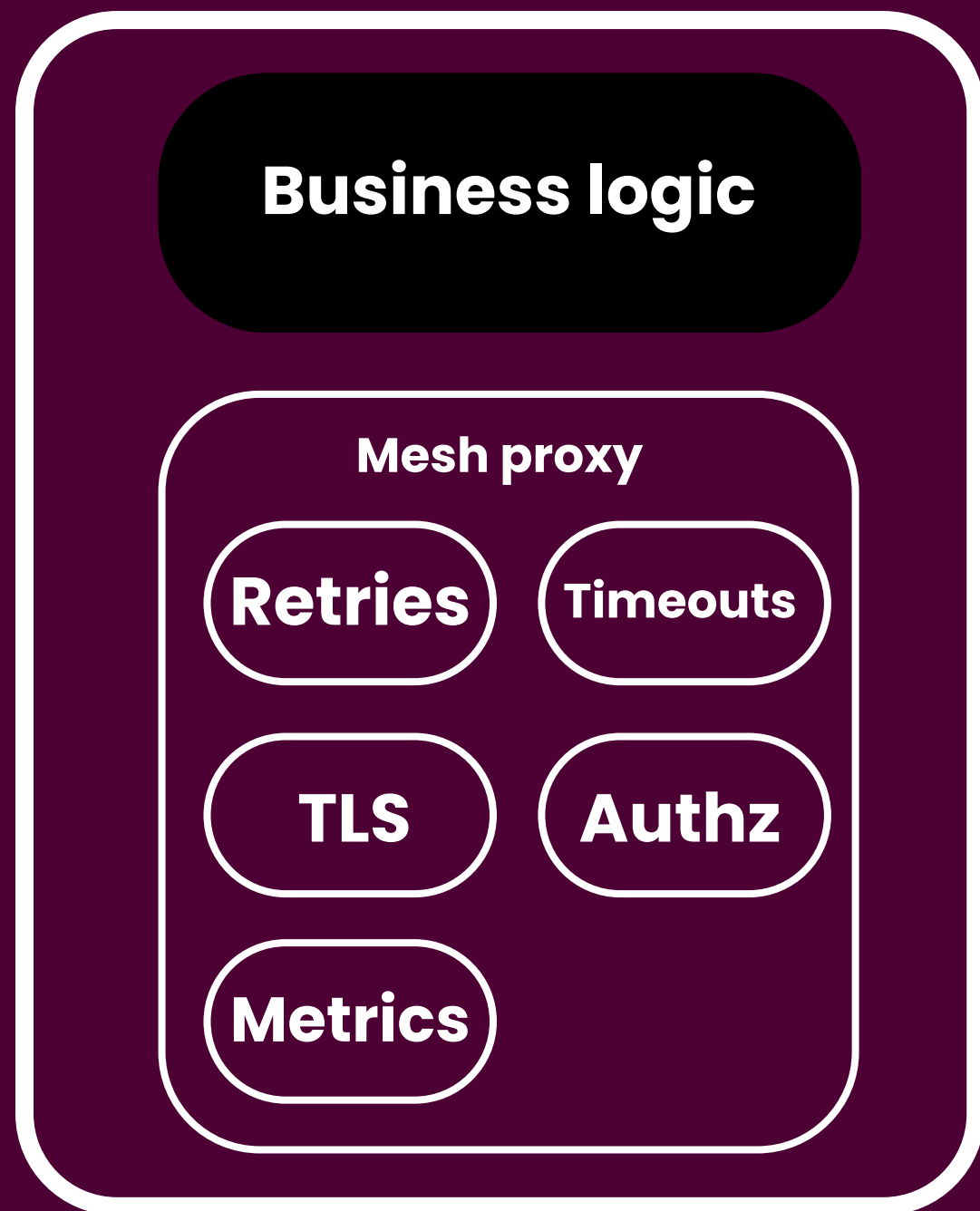
The service mesh concept

Networking, outside the app

Before



After



The app keeps business logic

The mesh handles security, traffic and telemetry

What is Istio

Istio is an open source **service mesh**:

one infrastructure layer that secures, routes and observes every call between services

Born in 2017

Launched by **Google** and **IBM**, built on **Envoy**, the proxy created at Lyft

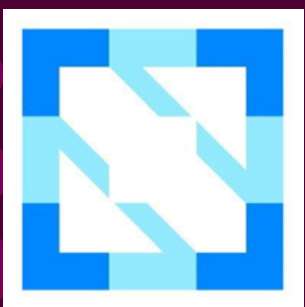


2022

Ambient mesh introduced, without sidecars. Joins the **CNCF** as **Incubating** (Sept 30)

2023

CNCF Graduated (July 12)

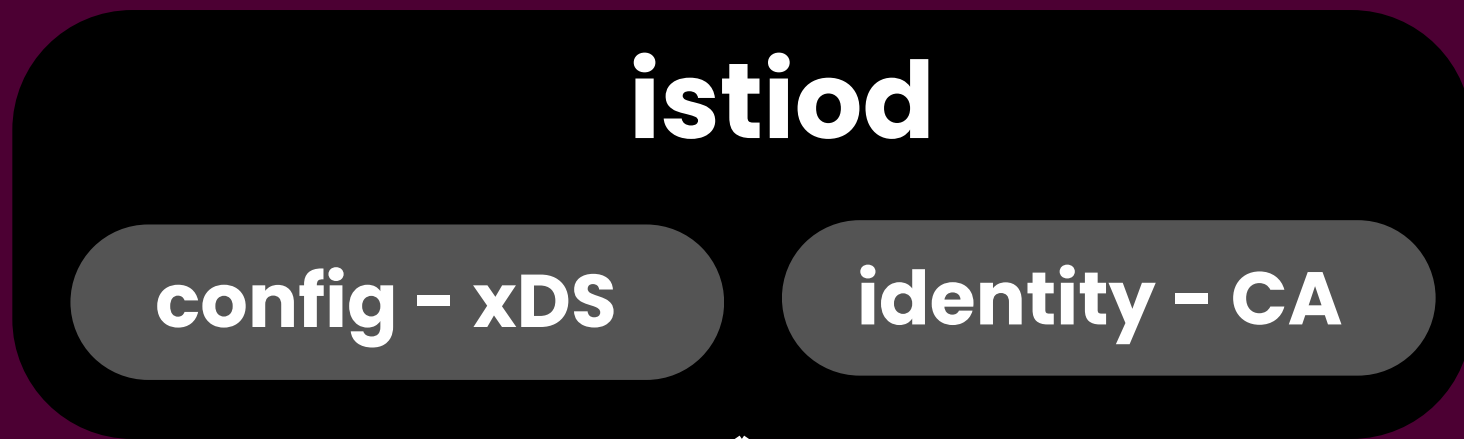


2024

Ambient mode GA in Istio 1.24

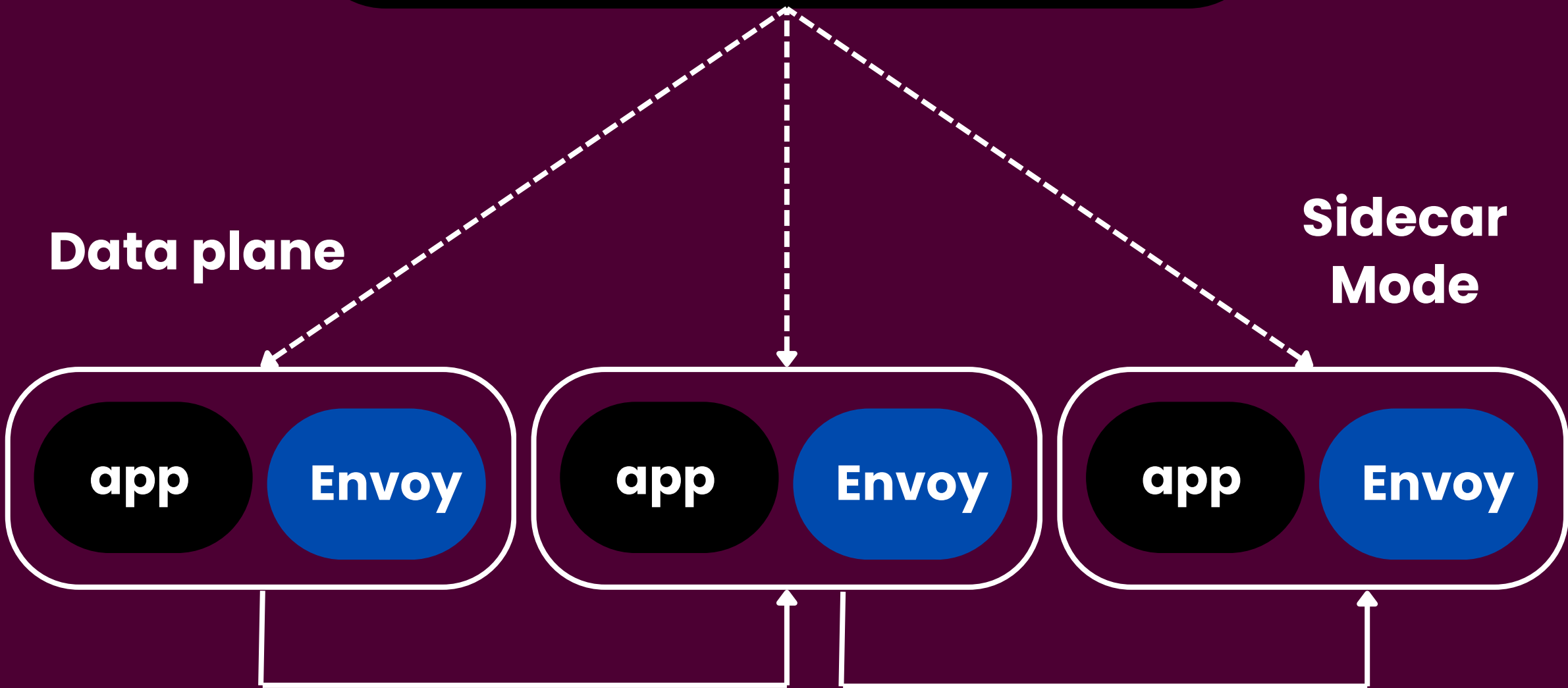
Architecture

Control plane



Data plane

**Sidecar
Mode**



mTLS between every proxy

You declare. istiod distributes

Proxies enforce

Zero trust

mTLS everywhere, in 8 lines



cert issued and rotated by istiod

peer-authentication.yaml

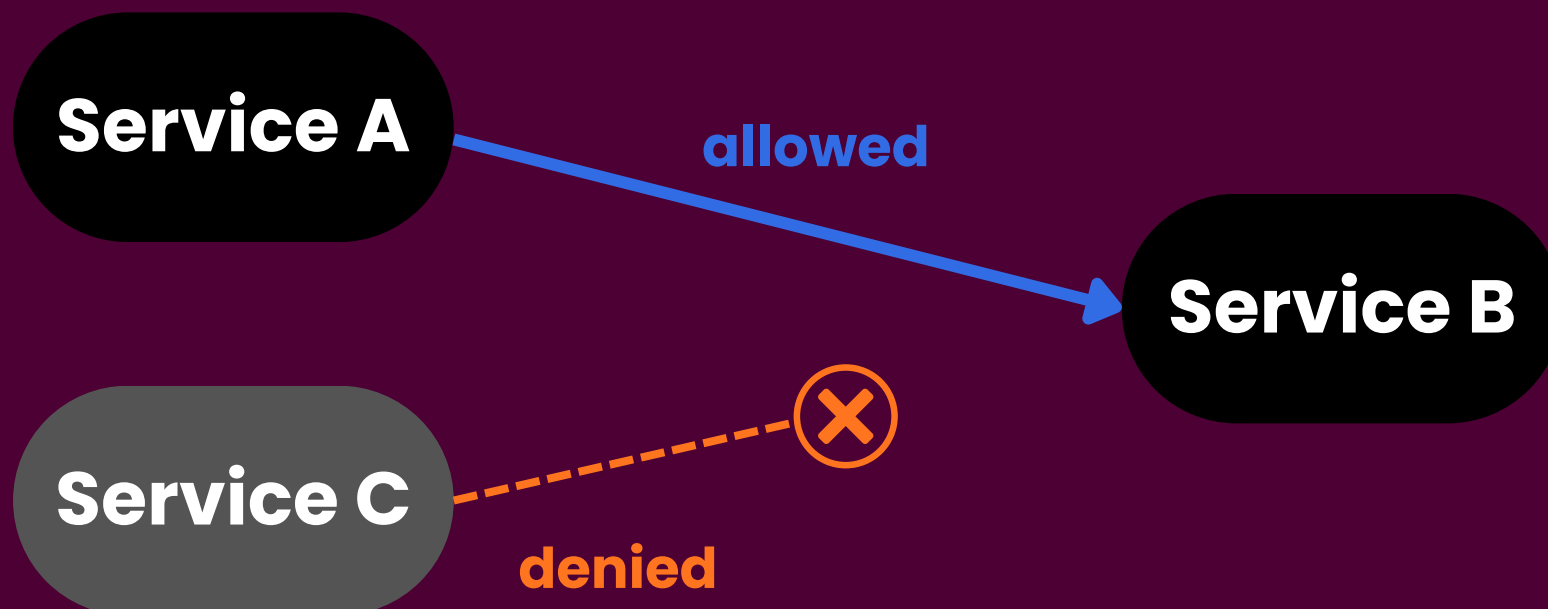
YAML

```
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: default
  namespace: istio-system # root ns = mesh-wide
spec:
  mtls:
    mode: STRICT
```

Mesh-wide: plaintext is refused

Zero trust

Only A may call B



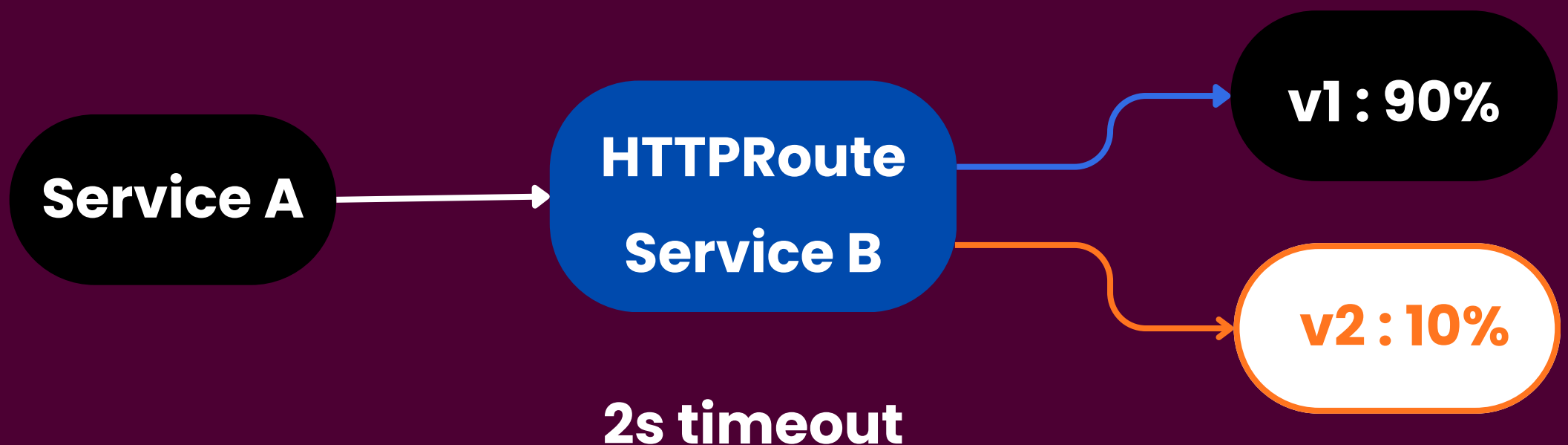
authorization-policy.yaml

YAML

```
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: service-b-allow-a
  namespace: trading
spec:
  selector:
    matchLabels:
      app: service-b
  action: ALLOW
  rules:
  - from:
    - source:
        principals:
        - cluster.local/ns/trading/sa/service-a
```

Traffic - Gateway API

Route 10% to v2



http-route.yaml

YAML

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: service-b
spec:
  parentRefs:
  - group: ""
    kind: Service
    name: service-b
    port: 8080
  rules:
  - backendRefs:
    - name: service-b-v1
      port: 8080
      weight: 90
    - name: service-b-v2
      port: 8080
      weight: 10
    timeouts:
      request: 2s
```

Observability



5xx on legacy-pricing, by caller

PROMQL

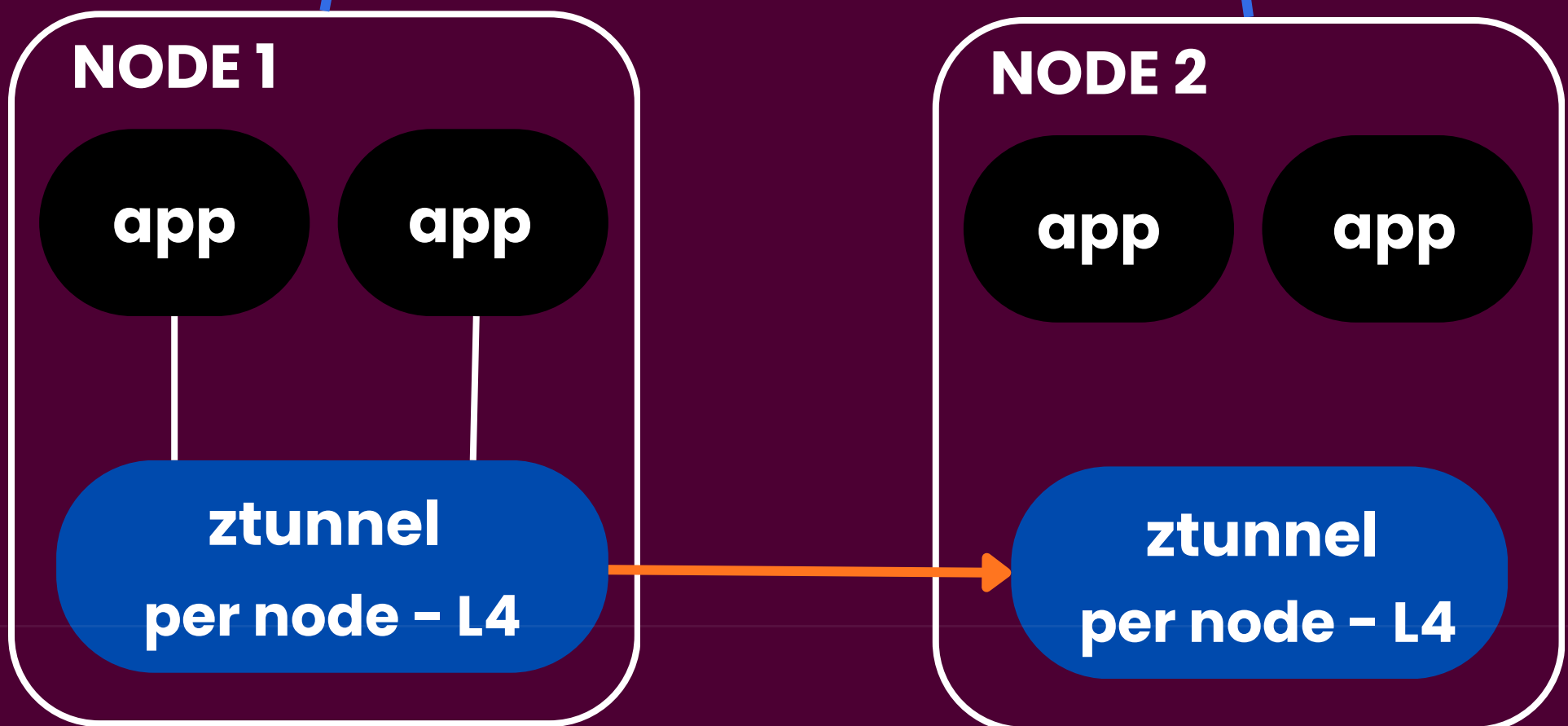
```
sum by (source_workload) (  
  rate(istio_requests_total{  
    destination_workload="legacy-pricing",  
    response_code=~"5.."   
  }[5m])  
)
```

Latency, traffic, errors:
the base for Service Level Objectives (SLO)

Ambient mode

No sidecars

Waypoint
optional - L7



HBONE - mTLS tunnel between nodes

```
enable ambient + L7 for a namespace
```

SH

```
kubectl label ns trading istio.io/dataplane-mode=ambient  
istioctl waypoint apply -n trading --enroll-namespace
```

L4 everywhere. L7 only where needed

Hands-on lab

Run it on your laptop

kind · Istio 1.31 ambient · Gateway API v1.6

STEP

4

mTLS, plaintext refused

STEP

5

Only service A may call service B

STEP

6

Ambient mode: add L7

STEP

7

Route 10% to v2

STEP

8

Metrics and Service Level Indicators

istio-ambient-lab

Link in the post